Step 2 — That Specialist, Now a Real Agent That Can Improve Its Own Work — Hints

Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What makes an agent able to improve
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What makes an agent able to improve

Two changes turn the Writer into an agent that can improve:

- it accepts **feedback** — what was wrong last time;
- something produces that feedback — a **Reviewer**.

Then a small loop: write → review → if not good enough, write again with the feedback → stop after a fixed number of rounds (Doc07's rule: never trust a loop to end on its own).

The Reviewer returns a checked object, not free text:

```
class ReviewVerdict(BaseModel):
    score: int = Field(ge=1, le=10)
    feedback: str
```

`run_reviewer(findings, draft)` = `prompt | get_model().with_structured_output(ReviewVerdict)` (Doc05).

Write the criteria into `reviewer.txt` — clarity, grounding (every fact has a `[source]`), completeness — and "7 or more only if all three are met". Without written criteria, a Reviewer approves almost everything, and the loop never runs.

The pass mark lives in code: `PASS_SCORE = 7` in `reviewer_agent.py`.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
write_with_review(findings) -> dict:
    result = {"draft": "", "accepted": False,
              "feedback_history": [], "revision_count": 0}
    feedback = ""
    for round_number in 1 .. MAX_REVISIONS:
        result["draft"] = run_writer(findings, feedback)
        verdict = run_reviewer(findings, result["draft"])
        if verdict.score >= PASS_SCORE:
            result["accepted"] = True
            return result
        feedback = verdict.feedback
        add feedback to result["feedback_history"]
        add 1 to result["revision_count"]
    return result
```

In `run_writer()`, turn an empty feedback into the word `"none"` so the prompt reads cleanly.

Why a dict with those exact keys? They are the names Step 5's shared team state will use — so `print_report(result)` keeps working unchanged when the graph replaces this loop.

`print_report(result)` must never show a failed draft as finished: if not accepted, print "Couldn't agree after N revisions", every piece of feedback, then the last draft marked **NOT approved**.

Test it two ways:

- `tests/integration/test_live_agents.py` — `run_reviewer(findings, "E-bikes are good.")` must score below `PASS_SCORE`.
- By hand — set `PASS_SCORE = 11` for one run and check the loop stops at the limit.

Full solution: Show me the solution